Information Fusion 118 (2025) 103017



Contents lists available at ScienceDirect

Information Fusion

journal homepage: www.elsevier.com/locate/inffus



Full length article



Anomaly detection for microservice system via augmented multimodal data and hybrid graph representations

Peipeng Wang, Xiuguo Zhang *, Zhiying Cao *

Information Science and Technology College, Dalian Maritime University, Dalian, 116026, Liaoning, China

ARTICLE INFO

Keywords:
Microservice system
Multimodal
Heterogeneous graph
Hard samples

ABSTRACT

Accurate anomaly detection is essential for ensuring the reliability of microservice systems. Current approaches typically analyze system anomaly patterns using single-modal data (i.e., traces, metrics, and logs) while neglecting the class imbalance between normal and abnormal samples, which can easily lead to misjudgment. This paper propose AMulSys, a graph-based anomaly detection approach, which adopts a hierarchical architecture to simultaneously model three modal data. First, we employ a unified graph structure to analyze the intricate scheduling relationships of traces and integrates various metrics to represent the system's resource consumption. Meanwhile, to capture the execution paths between logs in interactive services, we design a log heterogeneous graph modeling method, where the service to which the log event belongs is represented as a vertex attribute, and allows distinguishing the paths of the same and different services. Second, we propose a category prior guided multimodal data augmentation algorithm to alleviate class imbalance. It extends the Mixup to multimodal representations and incorporates category priors to bias the synthetic samples labels to the abnormal category. Furthermore, considering the impact of hard samples under class imbalance, we select and assign weights to hard samples, and perform multifaceted contrastive learning based on intra-/inter-modal and inter-sample to optimize pseudo sample. Evaluation results on two real microservice system datasets show that AMulSys outperforms state-of-the-art approaches and achieves an F1-score higher than 0.97.

1. Introduction

In complex microservice system, ensuring its stable operation has always been the primary task of developers due to its distributed and highly dynamic characteristics. Within the system, each microservice runs its own process while communicating with other services at various levels. If a service instance is affected by various factors such as misconfiguration, security attacks, software bugs, etc., it can lead to the failure of the entire system. Therefore, it is crucial to diagnose the system's faulty behavior in a prompt and precise manner [1].

Developers must continuously collect three types of monitoring data to identify potential issues and optimize system performance, including trace, log, and metric. As shown in Fig. 1, trace records the invocation path and response time to a user request, where each operation is called a span. Logs are used to describe the user behaviors and system security events. Metrics monitor the system's performance and resource usage [2]. To accurately detect system failures, recent researches have analyzed the trace response time and call structure alone [3], or the log semantics [4] or metric temporal feature [5] to perform automatic anomaly detection. Although they are effective, they still have two major limitations.

(1) Inadequate utilization of multimodal data. No single monitoring data can fully capture the system status. To prove this point, we selected five different types of anomaly cases from the open-source dataset . As shown in Table 1, When network latency occurs, trace response time increases, "Node Network Receive Bytes" goes up, but logs do not print an error message. Certain "Access exceptions" do not trigger any changes in the metrics. This phenomenon arises from the fact that the three types of monitoring data reflect the system status from different perspectives and have huge differences in data structure. The trace emphasizes the scheduling relationships between services and is represented as tree or graph data generated by the tracking system. The log offers detailed event records and contextual information, formatted as semi-structured text predefined by the programmer. The metric concentrates on quantitative performance and resource usage, regularly sampling time series data. In view of the drawbacks of unimodal anomaly detection, some studies have proposed multimodal anomaly detection methods. However, they lack a comprehensive analysis of the three types of data or lose high-level features (often trace structure or log semantics) within a certain modality. For example, UAC-AD [6] and Hades, only performed the interaction

* Corresponding authors.
E-mail addresses: zhangxg@dlmu.edu.cn (X. Zhang), czysophy@dlmu.edu.cn (Z. Cao).

https://doi.org/10.1016/j.inffus.2025.103017
Received 8 December 2024; Received in revised form 11 January 2025; Accepted 3 February 2025
Available online 11 February 2025
1566-2535/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

between metrics and logs. DeepTraLog [7] embedded log semantics as graph nodes of traces, capturing complex service interactions, but lacks analysis of metrics. AnoFusion [8] and MSTGAD [9] modeled logs as time series to analyze three types of data simultaneously, which avoids cross-modal differences but ignores log semantics. Therefore, it is challenging to extract comprehensive data features from each data type and effectively utilize complementary information of multimodal data.

(2) The class imbalance problem of normal and abnormal samples. The abnormal status during system operation only occurs occasionally, which results in normal monitoring data far surpassing abnormal data. In response to this imbalance problem, Ying et al. [10] employed clustering and self-training methods to improve the performance of KNN on imbalanced log sample sets. But this approach relies on the distribution of data and is sensitive to noise values. DiagFusion [11] manually increases the number of each type of fault case through event replacement. While this approach alleviates the issue of imbalance samples, it necessitates extensive expert knowledge and may result in poor generalization. Moreover, existing data augmentation strategies usually target single-modal data. It is essential to develop a robust and generalizable data augmentation algorithm in multimodal data-driven anomaly detection tasks.

In this work, we present AMulSys, an anomaly detection approach for microservice system via augmented multimodal data and hybrid graph representations. It jointly analyzes trace, metrics, and logs in one framework to achieve anomaly detection. Specifically, we establish a directed weighted graph based on invocation path of trace, and collect the metrics of the caller and callee services at the same time as vertex embedding. A Graph ATtention network (GAT) [12] is utilized to automatically develop the significance of various services to the detection results. At the same time, this paper attains a log heterogeneous graph, which takes log events of different services as vertexs and establishes discriminative edges for event pairs in the sequence. A Heterogeneous Graph Transformer Network (HGTN) [13] assigns distinct type parameters to depict vertexs and edges of varying services to encode heterogeneity. To address the problem of class imbalance, AMulSys applies the traditional Mixup algorithm [14] to the multimodal feature space and introduces class probability to improve the label mixing factor so that synthetic labels are biased towards abnormal category. It is noteworthy that, to improve the quality of pseudo data, we select and assign weights to hard samples prior to applying the data augmentation, perform intra- and inter-modal contrastive learning on them, followed by inter-sample contrastive learning after augmentation. Finally, we simultaneously calculate the labels of raw and pseudo samples to obtain the detection results.

The main contributions are summarized as follows:

- We propose a graph-based anomaly detection approach, AMulSys, to jointly analyze the three types of monitoring data within a framework and perform cross-modal interactions.
- We adopt two graph structures based on the characteristics of different monitoring data to comprehensively analyze the system status. This enables AMulSy to utilize the GAT to simultaneously analyze the inter-service invocation and the intra-service status, and utilize HGTN to capture the cross-service log context information.
- We devise a category prior guided multimodal data augmentation algorithm. It not only generates pseudo-samples with richer features from different modal feature spaces but also leverages label categories as prior knowledge, making the pseudo samples more inclined to abnormal cases, thereby addressing the issue of class imbalance.
- We develop a multifaceted contrastive learning based on intra-/inter-modal and inter-sample to learning cross-modal representation, in which we select and assign weights to hard samples to reduce computational overhead while ensuring data augmentation effects.

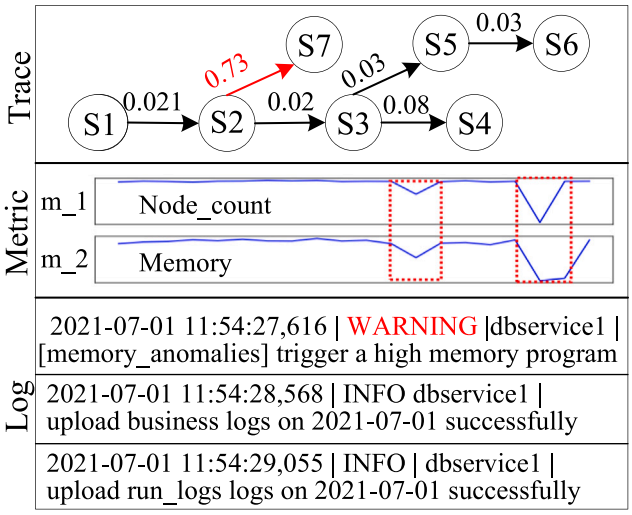


Fig. 1. A multimodal data case.

2. Related work

2.1. Single-modal data

Trace: According to the runtime information of the microservice system, corresponding trace fault detection methods were formulated. Earlier statistical methods such as TraceRCA [15] and MicroRank [16] performed detection by learning and comparing the mean, variance, standard deviation, and other statistical measures. But such methods were difficult to process large amounts of complex data. Considering the advantages of neural network, TraceAnomaly [17] utilized a VAE to simultaneously learn two types of abnormal situations: response time and call path of normal data. Nedelkoski et al. [18] proposed a multimodal LSTM network to perform structural anomaly detection and response time anomaly detection of trace. Compared to sequence models, graph neural networks have a natural advantage in analyzing trace structures. Therefore, TraceVAE [19] proposed a dual-variable graph VAE, which included structural VAE and temporal VAE, and calculated the negative logarithmic likelihood of a given trace as the anomaly score. To eliminate noise in the trace, TraceSieve [20] introduced an AE structure within an adversarial training framework, combined with VGAE and Elastic Weight Consolidation to perform anomaly detection from large-scale trace data. GRAND [21] combined VAE and GAN to detect abnormal trace. GTrace [3] grouped traces with the same substructure and utilized Tree-LSTM based VAE to obtain structural representations.

Log: Log parsing is the foundation of anomaly detection. Many researchers have explored parsing methods such as Drain [22], SwissLog [23], IPLoM [24], etc. Next, to analyze log events at different times, DeepLog [25] mapped log events as index, LSTM was utilized learn the normal index order. This indexing method is efficient, but it ignores log semantics, resulting in poor model robustness. To this end, LogBERT [26] utilized the BERT model to extract semantic features from logs. LogRobust [27] utilized the Fasttext with TF-IDF to generate log semantics, and combined LSTM and attention to detect abnormal logs. Considering the imbalance of log categories, LogEncoder [28] employed LSTM to encode log sequences and introduced contrastive learning to extract diverse features. ContexLog [4] retained all log information and input it into the Transformer encoder-based classification model. MLog [29] utilized Transformer encoder and IDF to obtain log semantics, and combines CNN and Mogrifer LSTM to capture log sequence representations from both global and local perspectives. Related to log graphs, LogGT [30] converted log sequences into graph structures and combined transfer learning methods for cross system log

Table 1
The anomaly cases of multimodal data.

Anomaly type	Trace	Log	Metric
network_delay	frontend-579b9bff58-t2dbm→ cartservice-579f59597d-wc2lz=2.5S	–	NodeNetworkReceiveBytes ↑
Connect error	frontend-579b9bff58-t2dbm→ frontend-579b9bff58-t2dbm=0.5	Error	–
Return failure	–	Error	CpuUsageRate↑
Access exception	frontend-579b9bff58-t2dbm→ productcatalogservice-668d5f85fb-wckp8=1.2	Error	–
RPC failure	frontend-579b9bff58-t2dbm→ currencyservice-cf787dd48-vpjrd==3.8s	Error	NodeNetworkReceiveBytes↑

analysis.

Metrics: As typical time series, metric-based fault detection can draw on a wide range of research related to time series. OmniAnomaly [31] utilized GRU based VAE to capture temporal features and calculate reconstruction scores. Xu et al. [32] employed Transformer to extract metric features. However, they ignore the correlation between different metric variables, which is of great significance to both detection results and practical applications [33]. To this end, MTAD-GAT [34] designed a graph learning layer to automatically learn the dependencies between variables and utilized GAT to extract spatial features. Zhu et al. [35] also used a graph structure, which established edges by calculating the covariance between different variables, and combined reconstruction and prediction methods for anomaly detection. MST-GAT [36] developed a multi-perspective GAT to learn spatial relationships of multi-sensor series, while a TCN was used to extract temporal features.

Existing methods analyze monitoring data from various perspectives and achieve impressive results. However, as shown in Table 1, no single modality can capture all types of anomalies, and Fig. 1 also shows significant differences exist among the various modalities. Therefore, AMulSys develops targeted modeling schemes for traces, logs, and metrics, obtaining modality-consistent and specific representations that reduce the discrepancies between modalities and provide a comprehensive description of the system's status

2.2. Multimodal data

With the advancement of fault detection technology and multimodal data analysis, [37] multimodal anomaly detection has gradually developed. Some studies integrate two types of monitoring data. SCWarn [38] and MULAN [39] modeled logs as time series and analyzed system status jointly with metrics. Hades [40] utilized Transformer to analyze log sequences, 1-D causal convolution to extract metric features, and a cross-modal attention was employed to achieve heterogeneous feature fusion. DeepTraLog [7] utilized graph structure to represent trace, treating logs as event sequences and using Glove to obtain log semantics. In each graph, log events were embedded as vertexs to train a GGNNs based deep SVDD model. TraceGra [41] provided a unified graph representation to combine trace and metrics, using a GAT to analyze the graph, and then using GCN and LSTM to extract structural and temporal features respectively. UAC-AC [6] introduced contrastive learning and adversarial training in metrics and logs to enhance the differentiation between normal and abnormal data. These methods can detect more anomalies than single-modal methods, but they are still insufficient to fully reflect the system status.

In recent multimodal anomaly detection, AnoFusion [8] serialized the three types of monitoring data, then generated graph flow representations based on MI, GTN is used to explore the association of different monitoring data, and GAT and GRU networks were used to detect fault states. MSTGAD [9], ART [42], DeepHunt [2] and UniDiag [43] also serialized logs and concatenated them with metrics as trace graph vertex embedding. Eadro [44] designed an end-to-end anomaly detection framework. It used the call relationship of trace to construct a dependency graph, and utilized GAT to capture the comprehensive system state for anomaly detection. However, different types of data have different ways of reflecting system states, and simple serialization and feature splicing are difficult to take advantage of multimodal data. Therefore, AMulSys employs a hierarchical architecture and develops two distinct graph structures to represent various monitoring data, utilizing a cross-modal attention mechanism to perform cross-modal interactions. DiagFusion [11] was the first to explore the class imbalance problem in microservice systems in the context of multimodal data, which expands the number of failure cases by manual replacement. However, this method relies on rich expert knowledge and is prone to introducing noise. Therefore, AMulSys designs a robust data augmentation algorithm based on multimodal features and incorporates contrastive learning to mitigate the effects of noise.

3. Motivation

Trace. In microservice system, user requests will involve multiple services, the entire request path can be intuitively displayed through the trace. Each span in the trace records information such as the response time and status code of the current service. When a failure occurs, the span will change. As shown in Fig. 2, we inject different faults into the normal trace, when a configuration error occurs in the microservice where Span E is located, Span D cannot call Span E correctly, which will manifest as a missing structure. In addition, when a database-related fault is injected, causing the database query performance of the service where Span D is located to degrade, the service will be blocked and the response time will be significantly beyond the normal range. These two typical faults are consistent with the exploration of trace in this work, when a system failure occurs, it will be manifested in the form of structural errors and response time anomalies, but the lack of fine-grained state information about the microservices in trace makes it easy to overlook potential anomalies.

Log. As shown in Fig. 3, "Log Event" is the parsed log template, where $\langle * \rangle$ represents variables. By extracting the log template semantics, we directly determine whether the current system activity is abnormal. Additionally, when injecting network latency faults into a normal log sequence, this can lead to changes in the order of execution in the log sequence. However, to ensure operational efficiency and code consistency, developers reuse the same codebase or modules in different services of the system, which results in different services containing the same log events. During anomaly detection, if the service to which the log template belongs in the current logging sequence is ignored, wrong order information may be obtained, which in turn leads to misjudgment. Moreover, logs may have a lag, and log analysis alone is not sufficient for microservice systems.

Metric: By monitoring key performance indicators and resource usage within different services, metrics help developers understand the operational status and performance of each service. Fig. 4 shows the "CpuUsage" of "webservice-S1" and "redisservice-S2" over a period of time. Obviously, "CpuUsage" indicates that the CPU is heavily occupied and in a depleted state through an irregular steep rise over a certain

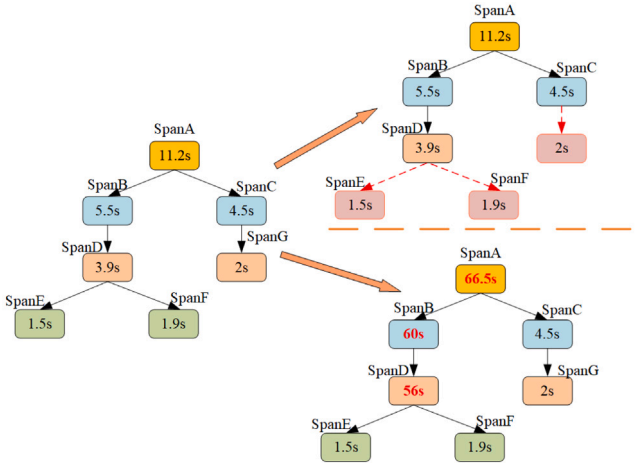


Fig. 2. Examples of anomaly trace.

period of time, and there is a dimensional difference in the distribution of "CpuUsage" between S1 and S2. These observation results are also consistent with the hypothesis of this work. Metrics can reflect the internal status of microservices at a fine-grained level, but it is difficult to directly analyze short-term cross-service metrics to describe the current system status.

4. Method

4.1. Overview

This paper proposes AMulSys, which employs a hierarchical architecture to analyze traces, metrics, and logs in one framework. As shown in Fig. 5, it mainly includes four modules: (1) Data preprocessing: Including structured trace, log parsing, metric normalization, and multimodal data alignment; (2) Inter-modal interaction: Extracting features from various modal data and performing cross-modal analysis; (3) Data augmentation and multifaced contrastive learning: A category prior guided multimodal data augmentation algorithm addresses the problem of class imbalance. A multifaceted contrastive learning to develop cross-modal representation and optimize pseudo sample.; (4) Anomaly detection and optimization. Performing anomaly detection based on augmented multimodal features, and calculating the loss of pseudo samples and raw samples.

4.2. Data preprocessing

Trace. In the original system dataset, trace is a message recorded when each microservice is invoked, which is stored in the form of a single invocation, so we first connect various microservices to form a tree structure of data based on the invocation relationship. Second, we filter out data in the trace that are too short, which are likely to be noisy interferences and do not contain enough information for modeling. In detail, for each trace x_i^t is composed of multiple Spans, which record multiple fields, including the jth service ID $id_{i,j}$, the time when the jth node receives the parent node message (start_time) $start_{i,j}$, the time when the message is returned to the parent node (end_time) $end_{i,j}$, etc.

Log. To obtain structured log events, we utilize the widely accepted Drain algorithm [22] for parsing. Subsequently, due to the updates of microservice systems, enterprises will adjust their log templates. To adapt to this change, we employ BERT [45] to analyze log event. BERT consists of multiple Transformer encoders that can process semantic information at different levels. To avoid word order loss, we take complete log events l_i instead of log words as input to obtain log semantics c_i. After preprocessing, AMulSys can obtain the service ID, timestamp, log template, log semantics.

Metric. The metric of microservice systems belong to typical multivariate time series. we first perform resampling and nearest neighbor missing value interpolation. Secondly, to eliminate dimensional differences between different variables within the same service, AMulSys performs maximum and minimum normalization. Formally, for the jth observation value $x_j^{m^i}$ ith variable x^{m^i}:

$$x_j^{m^i} = \frac{\tilde{x}_j^{m^i} - \min(\tilde{x}^{m^i})}{\max(\tilde{x}^{m^i}) - \min(\tilde{x}^{m^i})} \tag{1}$$

where $\min(\tilde{x}^{m^i})$ represents the minimum value of all observed values in variable $\tilde{x}^{m^i}$, $\max(\tilde{x}^{m^i})$ represents the maximum value. Due to the collection of the same metric variables in different microservices, after preprocessing, AMulSys can obtain the service ID, timestamp, and normalized value.

Aligned Multimodal Data. In the process, we first obtain the specific service ID involved in each trace, then query the start and end time of the service, and calculate the execution time $Et = end - start$. Next, we retrieve the metric values $(x_j^{m^i})$ and log events (l_i) for the corresponding time period under the same service ID. Therefore, each trace $x_i^t = (Et_{i,1}, Et_{i,2}, \ldots, Et_{i,n1})$, $n1$ is the number of span, the corresponding log sequence $x_i^l = (c_{i,1}, c_{i,2}, \ldots, c_{i,n2})$, $n2$ is the length of sequence, the corresponding metric series $x_i^m = (x_{i,1}^m, x_{i,2}^m, \ldots, x_{i,n1}^m)$, where $x_{i,j}^m = (x_{i,j}^{m^1}, x_{i,j}^{m^2}, \ldots, x_{i,j}^{m^p})$, p is the number of variables. It is worth noting that the metrics are sampled at fixed time intervals, while some microservices print multiple log messages in the current business. Therefore, in most cases, each span of the trace will correspond to one metric data and more log messages, $n2 > n1$. Subsequently, labels are assigned to each trace based on the fault injection time period. Finally, the we obtain the aligned multimodal data $\langle X, Y \rangle = \{(x_1^t, x_1^l, x_1^m, y_1), (x_2^t, x_2^l, x_2^m, y_2), \ldots, (x_n^t, x_n^l, x_n^m, y_n)\}$.

4.3. Inter-modal interaction

In this section, we conduct feature extraction and cross-modal interaction on multimodal data to leverage the joint information for subsequent data augmentation and anomaly detection.

4.3.1. Trace-metric representation

Trace-metric graph. In order to retain the information of trace structure completely, as shown in Fig. 6, we transform the trace into a directed weighted graph $G^T = \{V^T, E^T, W^T\}$, where V^T is the set of vertexs, each vertex represents a Span, $|V^T| = R$, R stands for the number of microservices in the system. W^T is the weight set. E^T is the set of edges, $e_{a,b}^T = (v_a^T, w_{ab}, v_b^T) \in E^T$, v_a^T is caller services, v_b^T is callee services, w_{ab} is weight. Different from existing studies [41], we define relative start time as the weight of the edge. The relative start time Rt_i is the difference between the current span's start_time $start_i$ and the root's start_time $start_{root}$. Analyzing the relative start times can help determine whether concurrent processing occurs among microservices, thereby providing a clearer understanding of the invocation order between them.

In order to refine the span in a cross-service global perspective, we collect the metrics of each span. For each vertex $v_i^T \in V^T$, the initial feature vector t_i^m is the time information of the service spliced with metrics representation x^m. Where time information includes not only execution time Et_i, but also waiting time Wt_i, local execution time Lt_i.

$$Wt_i = \sum_{j \in 'i-child} (end_j - start_j), Lt_i = Et_i - Wt_i \tag{2}$$

where i - child is the set of called services and is represented as a child of vertex i in the structure graph. These time information, including Rt_i, also need to be normalized. Therefore, AMulSys achieves cross modal fusion of trace and metric through a directed weighted graph.

GAT. As mentioned earlier, the structure and response time of trace in microservice systems directly determine whether the system

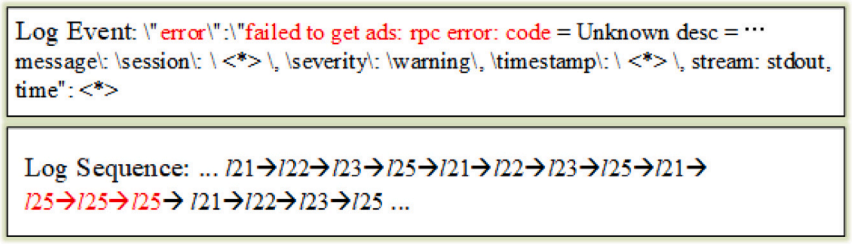


Fig. 3. Examples of anomaly log.

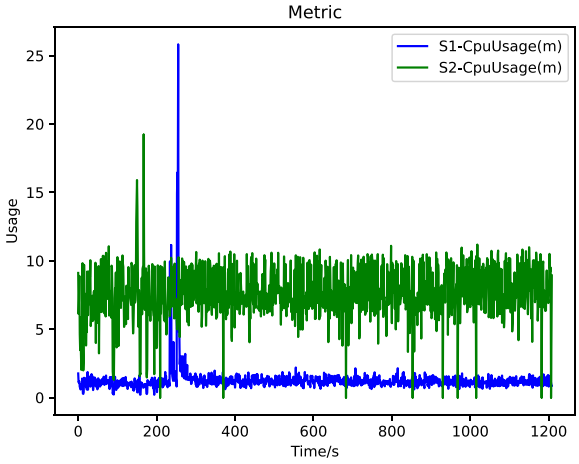


Fig. 4. Examples of anomaly metric.

is abnormal. In order to effectively learn the time information and metric embedding in the graph, AMulSys employs a GAT on G^T to calculate the importance of different span for anomaly detection results and capture the system state. Specifically, in the trace-metric graph, vertex i feature $t_i^m \in \mathbb{R}^c$, $c = p + 3$, passes through the GAT layer and outputs its hidden feature h_i^t.

$$h_i^t = \phi(\sum_{j \in V} \alpha_{ij} W t_j^m) \tag{3}$$

where $h_i^t \in \mathbb{R}^{d_t}$ incorporates information from all neighboring vertexs j, ϕ represents the ReLU activation function. $W \in \mathbb{R}^{d_t \times c}$ is a learnable parameter matrix, which is used to perform linear transformations on the input to obtain discriminative vertex representation. α_{ij} is the attention score of vertex j to i, which is obtained by normalizing the attention coefficient e_{ij} between the two vertexs. It is worth noting that we also considered the weight w_{ij} of the edges.

$$e_{ij} = \text{LeakyReLu}(\mathbf{a}^T \cdot [W t_i^m \| W t_j^m \| w_{ij}]) \tag{4}$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in V} \exp(e_{ik})} \tag{5}$$

where LeakyReLU is activation function. $\mathbf{a} \in \mathbb{R}^{2 \times d_t + 1}$ is learnable column vector. Finally, AMulSys performs global attention pooling to obtain a trace-metric representation h^{tm}.

4.3.2. Log representation

Log heterogeneous graph. A shown in Fig. 7, to determine the source of each log event and avoid obtaining the wrong execution path leading to misjudgment, we extract the microservice to which each log event belongs, form a service sequence and design a novel log heterogeneous graph $G^L = \{V^L, E^L, T^v, T^e\}$, where V^L is the set of vertexs representing different log templates l, T^v is a set of vertex types. In Fig. 7, we utilize various colors to represent different microservices, allowing for the clear distinction of various types of nodes. E^L is a set of edges, each edge $e = (s, t, w_{st})$ contains a source event s, a target event t and a weight w_{st} whose value is the frequency of the event pair $s \to t$ in the sequence. T^e is a set of edge types. In Fig. 7, solid and dashed lines are used to represent the execution relationships both within and across services.

Subsequently, since the original log sequence is generated based on the invocation relationships among different microservices, the intra-service workflows may be overlooked, and some anomalies in the system may be caused by precisely these internal workflow issues. For example, when processing a user's payment request, it will include order service, payment service, etc. One failure may be that the user did not receive a notification of successful payment. Only analyzing the cross-service log sequence will find that all calls have responded successfully. However, further analysis of the payment service will find that due to network delays, the log execution order within the service does not conform to the normal mode. Therefore, AMulSys further establishes edges for log events of the same service that do not have a call relationship according to the execution order, as shown in $l7 \to l6$ in Fig. 7 .

HGTN. In order to capture the distinguishing features of various types of vertexs and edges in the log heterogeneous graph, we introduce a heterogeneous GTN [13], which obtains the target event t features by aggregating the information of different source events s. The entire process is shown in Fig. 8, AMulSys represents the log semantics as an vertex embedding h_0^g, defines the $l-1$ layer output of s to be $h_{s,l-1}^g$ and to be used as the l layer input. Firstly, s is mapped to the ith key vector $K^i(s) \in \mathbb{R}^{\frac{d_g}{h}}$ and t is mapped to the ith query vector $Q^i(t) \in \mathbb{R}^{\frac{d_g}{h}}$ using the different linear layers $Linear^i_{\gamma(s)}$, where $\gamma(s)$ is the type of s, i.e. the service to which it belongs, for modeling heterogeneous features. Then, we employ dot product to compute the similarity between $Q^i(t)$ and $K^i(t)$, and combine the weights w_{st} of the edges as the attention scores $head^i(s, e, t, w_{st})$ of the ith heads.

$$head^i(s, e, t, w_{st}) = (K^i(s) W^h_{\varphi(e)} Q^i(t) \cdot w_{st}) \cdot \frac{\mu\langle\gamma(s), \varphi(e), \gamma(t)\rangle}{\sqrt{d}} \tag{6}$$

where $\varphi(e)$ is the edge type mapping function: $E^L \to T^e$. So as to extract different meta-relationships and further distinguish log events from various services, we add an a priori tensor containing vertex, edge types $\mu\langle\gamma(s), \varphi(e), \gamma(t)\rangle$. $W^h_{\varphi(e)} \in \mathbb{R}^{\frac{d_g}{h} \times \frac{d_g}{h}}$ is a trainable adjacency matrix of different types of edges. For the target event t, we connect h attention heads to obtain its attention vector $Att(s, e, t)$.

Subsequently, HGTN models the information transmission process from the source event to the target event. The hidden feature $h_{s,l-1}^g$ of the source event s in the earlier layer is used as input, and a linear layer combined with the vertex type $\gamma(s)$ is also used to calculate the message vector $M_head^i(s, e, t)$ of the ith information header.

As illustrated in Fig. 8, the message vector $M_head(s, e, t)$ for each vertex pair is obtained by aggregating h message headers. Based on the attention score $Att(s, e, t)$, HGTN weights and aggregates the message vectors from the source event to update the target vector.

$$\tilde{h}_t^g = \bigoplus_{\forall s \in N(t)} (Att(s, e, t) \cdot M_head(s, e, t)) \tag{7}$$

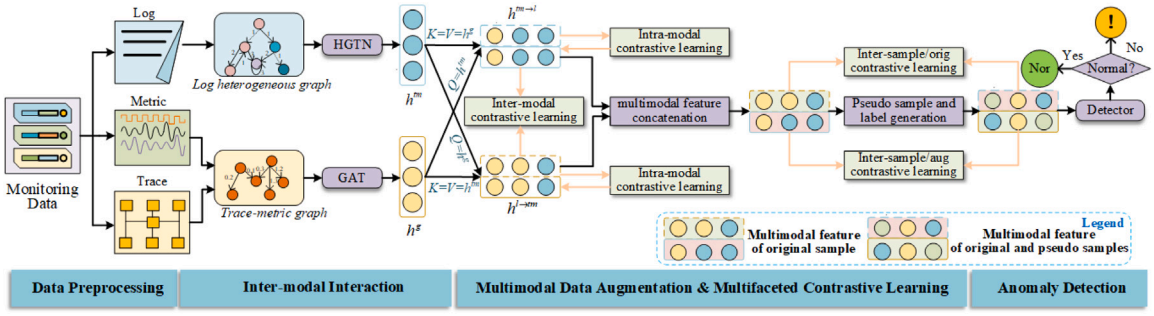


Fig. 5. The framework of AMulSys.

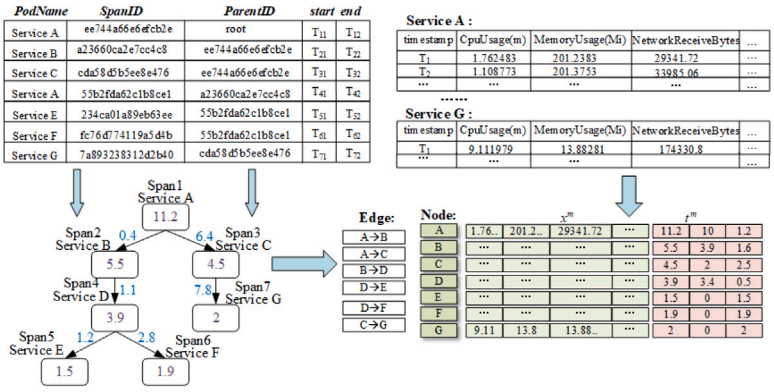


Fig. 6. An example of the trace-metric graph.

In this way, the message vectors of all source events from different services are aggregated to the target event. $\oplus$ represents feature aggregation. When obtaining the target event output, we map it back to a specific service distribution through a linear layer $A_Linear_{\gamma(t)}$ and further enrich the target output, i.e. $h^g_{t,l}$, through residual connections.

Finally, the output features are obtained by stacking L layers of networks. In an effort to reduce the computational complexity and avoid information loss, we aggregate all event representations by average pooling to obtain the log representation h^g.

It is worth emphasizing that HGTN parameterizes the matrix by introducing a specific type of meta-gate $\langle \gamma(s), \varphi(e), \gamma(t) \rangle$ and adaptively adjusts the vertex and edge relationships, which is not only conducive to processing the distribution differences of heterogeneous graphs, but also avoids the need for traditional manual setting of meta-paths, thereby more flexibly capturing the key information of heterogeneous graphs.

4.3.3. Time complexity

AMulSys employs GAT to model the trace-metric graph. Assuming that the number of nodes in the graph is R and the dimension of the feature vector is c, the time complexity of calculating the attention coefficients for each node v_i in each layer of GAT is $O(c \cdot R)$, while the complexity of computing the weighted values of neighboring nodes is $O(R)$. Consequently, the total time complexity for all nodes is $O(c \cdot R^2)$. Therefore, the time complexity of GAT is primarily influenced by the number of spans and metric dimensions, without incurring additional computational overhead.

Furthermore, HGTN is utilized to model log heterogeneous graphs. Assuming that the number of log events is N, the dimension of the embedding space is d_g, the linear mapping complexity of the node is $O(N \cdot d_g^2)$. The calculation of the attention score requires traversing all neighboring nodes, the time complexity is $O(h \cdot N^2 \cdot d_g)$, h is the number of attention heads. The time complexity of information transmission and aggregation is $O(h \cdot D \cdot N \cdot d_g)$, where D represents the number of neighboring nodes. The time complexity of feature updates is $O(D \cdot N \cdot d_g)$. Therefore, the overall time complexity is $O(h \cdot D \cdot N \cdot d_g + N \cdot d_g^2)$. It is evident that HGTN incurs greater computational overhead because modeling heterogeneity necessitates consideration of the types of nodes and edges. In practical applications, optimizing the number of attention heads and reducing the feature dimension can effectively decrease the computational load.

4.3.4. Cross-modal attention

In order to alleviate the differences in representations of different modalities and make full use of the complementary information of various monitoring data, the paper introduces a cross-modal attention mechanism [40], which consists of two layers of attention. The first one takes the trace-metric feature h^{tm} and log feature h^g as input. It utilizes a linear layer to map h^{tm} to the query vector q^{tm} and maps h^g to both the key vector k^g and value vector v^g, so as to learn the log context under the guidance of the trace-metric. The calculation is as shown in Eq. (8).

$$Att = \text{softmax}(\frac{q^{tm}(k^g)^T}{\sqrt{d_k}})v^g \tag{8}$$

where d_k is the dimension of k^g. Finally, the log representation $h^{tm \to l} \in \mathbb{R}^{d_g}$ is output through residual connection and normalization.

$$h^{tm \to l} = LayerNorm(Att + h^g) \tag{9}$$

where $LayerNorm$ represents layer normalization. Similarly, the second attention layer maps h^g to the query vector q^g, while h^{tm} is mapped to both the key vector k^{tm} and the value vector v^{tm}. We can learn the trace-metric representation under the guidance of the log and obtain $h^{l \to tm} \in \mathbb{R}^{d_t}$. It is worth mentioning that, through the calculation of attention scores, we can also identify the key features in h^{tm} and h^g that are crucial for detection results, which is conducive to adapting to changes in system status.

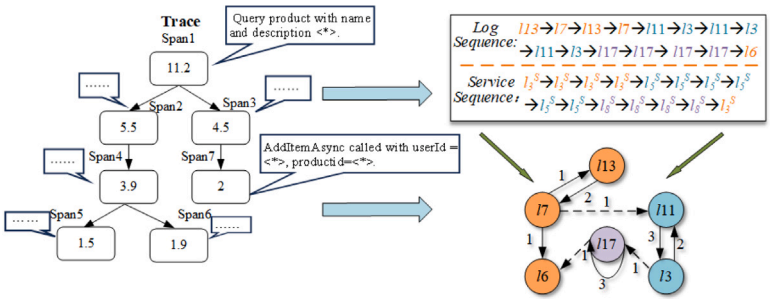


Fig. 7. An example of the log heterogeneous graph.

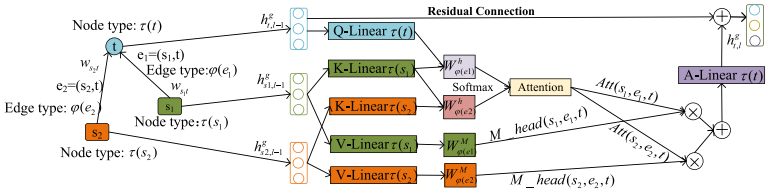


Fig. 8. Log heterogeneous graph representation based on HGTN.

4.4. Category prior guided multimodal data augmentation

Traditional data augmentation algorithms, such as Easy Data Augmentation (EDA) [46], do not require extensive computing resources, but they can compromise the original semantics of multimodal data. While SMOTE (Synthetic Minority Over-sampling Technique) is widely used, it struggles to effectively manage the relationship between trace-metric representation and log representation. Additionally, some generative models, like Generative Adversarial Networks (GANs), possess strong generative capabilities, they demand significant computing resources. This paper selects the Mixup algorithm, which is both robust and computationally efficient, to alleviate the class imbalance problem of multimodal monitoring data. The Mixup [47] idea was originally proposed in the image field. Later, Manifold Mixup [14,48] was proposed to generate pseudo samples in the embedding space. In order to further distinguish the content and style features of the image, StyleMix [49] was proposed to improve the model's generalization capability. Motivated by these, AMulSys proposes a multimodal data augmentation algorithm to enrich anomaly samples, relieving model's bias towards normal class.

We first randomly select monitoring sample pairs (x_1, y_1), (x_2, y_2) from a mini-batch. The trace-metric representation (Tm_r) $h_1^{tm\to l}$, $h_2^{tm\to l}$ and the log representation (L_r) $h_1^{l\to tm}$, $h_2^{l\to tm}$ are obtained following inter-modal interaction. In order to enrich the sample features, we perform multimodal feature fusion inter- and intra-samples:

$$h^{11} = [h_1^{tm\to l}; h_1^{l\to tm}], h^{22} = [h_2^{tm\to l}; h_2^{l\to tm}], h^{21} = f(h^{11}, h^{22}),$$
$$h^{12} = f(h^{22}, h^{11}) \tag{10}$$

where $[;]$ represents feature splicing. $f(h^{ij}, h^{jj}) = \sigma(h^{jj})\left(\frac{h^{ij}-\mu(h^{ij})}{\sigma(h^{ij})}\right) + \mu(h^{jj})$ is a normalization layer. [49] $\mu(\cdot)$ and $\sigma(\cdot)$ represent the mean and variance, respectively. h^{11} and h^{22} are multimodal features intra-sample, h^{12} and h^{21} are multimodal features inter-sample. Secondly, we set two factors, Tm_r ratio λ_t and L_r ratio λ_l to execute linear interpolation on the four features. Unlike traditional Mixup algorithms [47], the mixed feature $\tilde{h} \in \mathbb{R}^{d_t+d_g}$ contains λ_t of $h_1^{tm\to l}$, $1-\lambda_t$ of $h_2^{tm\to l}$, λ_l of $h_1^{l\to tm}$ and $1-\lambda_l$ of $h_2^{l\to tm}$.

$$\tilde{h} = \gamma h^{11} + (\lambda_t - \gamma)h^{12} + (\lambda_l - \gamma)h^{21} + (1 - \lambda_t - \lambda_l + \gamma)h^{22} \tag{11}$$

where λ_t and λ_l are obtained by sampling with $\mathrm{Beta}(\alpha, \alpha)$, α is a hyperparameter. MulSys introduces the free variable $\max(0, \lambda_t+\lambda_l-1) \le \gamma \le \min(\lambda_t, \lambda_l)$, and controls it within the allowable range, which not only dynamically adjusts the contributions of $h^{tm\to l}$ and $h^{l\to tm}$ in the pseudo samples, but also avoids introducing redundant information.

Meanwhile, we initially synthesize pseudo-sample label $\tilde{y}'$ based on Tm_r radio λ_t and L_r radio λ_l.

$$\tilde{y}' = \lambda_t y_1 + (1-\lambda_t)y_2 + \lambda_l y_1 + (1-\lambda_l)y_2 \tag{12}$$

Then, in order to further expand the low-frequency anomaly samples during the training process, AMulSys introduced category probability as prior knowledge, which guides the model to assign a larger mixing factor to abnormal labels, making the synthesized pseudo samples biased towards a small amount of abnormal data. The category mixing factor λ_r can be obtained by the following formula:

$$\lambda_r = 1 - \tanh\left(\frac{p(x_1)}{p(x_1)+p(x_2)}\right) \tag{13}$$

where $p(x_1)$ and $p(x_2)$ represent the class probabilities corresponding to samples x_1 and x_2, respectively, According to the formula, the higher the probability of category x_1, the smaller λ_r will be.

Then, the new label mixing factors λ_{rt} and λ_{rl} are obtained by averaging the category mixing factors λ_r with Tm_r radio λ_t and L_r radio λ_l, respectively.

$$\lambda_{r\mu} = \frac{2\lambda_r\lambda_\mu}{\lambda_r + \lambda_\mu} \tag{14}$$

Where $\mu \in \{t, l\}$. Finally, we obtain pseudo sample label.

$$\tilde{y} = \lambda_{rt} y_1 + (1-\lambda_{rt})y_2 + \lambda_{rl} y_1 + (1-\lambda_{rl})y_2 \tag{15}$$

4.5. Multifaceted contrastive learning

In order to prevent pseudo samples from "blurring" the feature boundaries and to explore complex relationships in multimodal data, we construct pairs of positive and negative samples and attain contrastive losses from both intra/inter-modal and inter-sample perspectives. This allows us to fully leverage supervised information to learn the consistency across modalities and the distinctions between the normal and abnormal status of the system, mainly including two aspects: before and after data augmentation.

Intra- and inter-modal contrastive learning. This process is conducted prior to data augmentation, utilizing the interacted cross-modal representations $h^{tm\to l}$ and $h^{l\to tm}$ as input. It aids the model in learning

more discriminative features both intra- and inter modalities, thereby providing high-quality input for subsequent data augmentation.

We first introduce intra-modal contrastive learning. Considering the complex operating environment of microservice systems, various external factors, such as network fluctuations, resource constraints, and changes in user behavior, can disrupt the system's status. These factors can cause variations among samples within the same modality. Intra-modal contrastive learning utilizes samples from the same class and modality as positive pairs [50], while negative pairs are defined as samples from different classes within the same modality to adapt to the dynamic system environment.

$$L_{tra} = -\frac{1}{M_P}\sum_{j=1}^{M_p}\log\frac{e^{sim(h_i^a,h_j^a)/\gamma}}{e^{sim(h_i^a,h_j^a)/\gamma}+\sum_{k=1}^{M_N}e^{sim(h_i^a,h_k^a)/\gamma}} \tag{16}$$

where $a \in \{tm \rightarrow l, l \rightarrow tm\}$, h_j^a and h_k^a represent the positive and negative instances of h_i^a, respectively. M_P and M_N represent the number of positive and negative instances respectively. A mini-batch of size M, then $M-1 = M_N + M_P$ to prevent self-computation. γ is the temperature coefficient.

In real microservice systems, traces, logs, and metrics work together to present a comprehensive view of the system's status. The cross-modal attention addresses the differences between modalities from within the same sample perspective, while inter-modal contrastive learning focuses on extracting complementary information from different modalities across various samples. It regards different modal representations of the same class as positive pairs and different modal representations of different classes as negative pairs.

$$L_{ter} = -\frac{1}{M_P}\sum_{j=1}^{M_p}\log\frac{e^{sim(h_i^a,h_j^{da})/\gamma}}{e^{sim(h_i^a,h_j^{da})/\gamma}+\sum_{k=1}^{M_N}e^{sim(h_i^a,h_k^{da})/\gamma}} \tag{17}$$

where $da \in \{tm \rightarrow l, l \rightarrow tm\}$, $a \neq da$ to distinguish different modalities. Through inter-modal contrast learning, the model can better understand the consistency of the current system status in different monitoring data. Furthermore, considering the unique perspectives offered by various monitoring data regarding the system status which should be retained to provide modality-specific information. We further introduce a regularization formula in the positive pair to prevent excessive similarity between modalities:

$$L^R = \frac{1}{M_p}\sum_{j=1}^{M_p}\|sim(h_i^a, h_j^{da}) - \delta\|^2 \tag{18}$$

L_{ter} is updated to $L_{ter} + L^R$. δ is the key parameter used to control the modal gap [51]. If δ is adjusted upward, $sim(h_i^a, h_j^{da})$ becomes larger and modality-specific insight may be lost.

Hard sample selection and weighting strategy. In addition to class imbalance, there are some hard samples in the dataset, which may be generated by service dependency changes, log event updates, or user behavior changes. Hard samples manifest as normal data that is situated near abnormal patterns, or as abnormal data that tends to resemble normal status. Consequently, they appear as low-similarity positive pairs and high-similarity negative pairs in the feature space. Previous studies [52] have concentrated on the weighting of hard negative samples, as they may affect the effectiveness of obtaining negative samples of the same class. However, during the anomaly detection process, applying data augmentation to hard positive samples (abnormal samples) can also produce more ambiguous pseudo-abnormal data, which may hinder the model's discriminability [53]. To address this issue, we assess the "degree" of the samples in a supervised manner, select both hard positive samples and hard negative samples from the unbalanced dataset, and dynamically allocate weight factors based on their degree.

Specifically, AMulSys utilizes similarity functions corresponding to contrastive learning to define the degree of positive and negative samples respectively, thus naturally combining contrastive loss. Subsequently, for sample a, select the sample pairs with high degree for training.

$$D^P = Min_k\left(sim(a,P)\right), D^N = Max_k\left(sim(a,N)\right) \tag{19}$$

where D^P and D^N are the degree sets obtained from the calculation. P and N are sets of samples of the positive and negative from a, respectively. In contrastive losses, they can come from the same modality and different modalities. Min_k and Max_k are the k samples with the lowest and highest similarity, respectively. During the training process, the computational cost of contrastive loss depends on the size M of the mini-batch. If M is set to be large, the number of sample pairs increases and the computational cost increases. Now, AMulSys only calculates k (k can be freely adjusted) hard samples, thereby reducing the overhead without introducing additional parameters. Next, we calculate the weight factors α_i^N and α_i^P of the ith hard sample according to the degree.

$$\alpha_i^P = 1 - \frac{\exp(D_i^P)}{\sum_{i=1}^{k}\exp(D_i^P)}, \alpha_i^N = \frac{\exp(D_i^N)}{\sum_{i=1}^{k}\exp(D_i^N)} \tag{20}$$

It is evident that AMulSys assigns greater weights to positive samples with low similarity and to negative samples with high similarity. Then, the similarity score of the contrastive loss is updated with the weight factor.

$$s_i^N = D_i^N \cdot (\alpha_i^N)^{\varphi}, s_i^P = D_i^P \cdot (\alpha_i^P)^{\varphi} \tag{21}$$

$$\varphi = \begin{cases} \lg N(nor)/N(abnor), N(nor)/N(nor) > 10 \\ 1, N(nor)/N(nor) \leq 10 \end{cases} \tag{22}$$

where $N(nor)$ and $N(abnor)$ represent the number of normal and abnormal samples in the dataset, φ is the scaling factor determined by the sample ratio, and it is used to control the weighting rate. Even if the k hard samples contain simple samples, the reduction ratio can be increased while avoiding overfitting of difficult samples.

Inter-sample contrastive learning. This process occurs after data augmentation. AMulSys conducts contrastive learning on the pseudo normal and abnormal samples alongside the raw normal and abnormal samples to ensure that the generated pseudo samples preserve the distribution of the raw samples while also providing sufficient diversity and discrimination [54]. Assume that, in the original mini-batch, there are M_{nor} normal samples, $M - M_{nor}$ abnormal samples. After data augmentation, the number of pseudo-normal samples is M'_{nor}, and the number of pseudo-abnormal samples is $M - M'_{nor}$. The inter-sample contrastive loss is determined by the original loss and the augmented loss:

$$L_{orig} = -\frac{1}{M'_{nor}}\sum_{j=1}^{M'_{nor}}\log\frac{e^{sim(h_i^{nor},\tilde{h}_j^{nor})/\gamma}}{e^{sim(h_i^{nor},\tilde{h}_j^{nor})/\gamma}+\sum_{k=1}^{M-M'_{nor}}e^{sim(h_i^{nor},\tilde{h}_k^{abnor})/\gamma}} \tag{23}$$

$$L_{aug} = -\frac{1}{M_{nor}}\sum_{j=1}^{M_{nor}}\log\frac{e^{sim(\tilde{h}_i^{nor},h_j^{nor})/\gamma}}{e^{sim(\tilde{h}_i^{nor},h_j^{nor})/\gamma}+\sum_{k=1}^{M-M_{nor}}e^{sim(\tilde{h}_i^{nor},h_k^{abnor})/\gamma}} \tag{24}$$

Among them, the two contrastive losses share the common goal of maintaining consistency between the raw normal samples and the pseudo normal samples. The difference lies in that L_{orig} emphasizes reducing the spatial distance between the raw normal samples and the pseudo abnormal samples, while L_{aug} focuses on minimizing the spatial distance between the pseudo normal samples and the raw abnormal samples. AMulSys does not further narrow the gap between pseudo-abnormal samples and raw abnormal samples. This is because the occurrence of system anomalies is influenced by numerous factors, rendering it both random and complex. Attempting to narrow this gap may lead the model to fit specific anomalies, ultimately diminishing its generalization ability.

It is important to note that the robustness of anomaly detection in microservice systems is primarily determined by the model's capacity to accurately understand the raw status represented by multimodal data following system updates or external interference [55]. To achieve

this, this paper simulates these actual changes by performing linear interpolation and feature fusion in the multimodal feature space, and ensures that the model can effectively differentiate between the normal and abnormal status of the system reflected by pseudo samples through multifaceted contrast learning.

4.6. Anomaly detection

We formalize anomaly detection as a classification problem by utilizing enhanced multimodal features, which include both raw samples and pseudo samples as input, and MLP containing Softmax to output the classification results.

Subsequently, the cross-entropy loss function was used to train AMulSys. In this process, we further incorporate data augmentation strategies into the deep learning anomaly detection model, and calculate the loss of true samples and pseudo samples respectively:

$$L_{cls} = \sum_{i=1}^{M} \left[-\left(y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right) \right] + \sum_{j=1}^{M} \left[-\left(\tilde{y}_j \log(\hat{y}_j) + (1 - \tilde{y}_j) \log(1 - \hat{y}_j) \right) \right] \tag{25}$$

where M is the size of mini-batch. $\hat{y}_i, \hat{y}_j \in \{0, 1\}$ are the prediction labels for the raw sample and pseudo sample, $y_i, \tilde{y}_j \in \{0, 1\}$ are the actual labels. Therefore, the former is the loss of original monitoring data, and the latter is the loss of pseudo samples.

We combine the contrastive loss and classification loss to form the overall loss function of AMulSys.

$$L = L_{cls} + \lambda_1 L_{tra} + \lambda_2 L_{ter} + \lambda_3 L_{orig} + \lambda_4 L_{aug} \tag{26}$$

where λ_1, λ_2, λ_3 and λ_4 is hyperparameters to control the contribution of the four contrastive losses. Algorithm 1 outlines the complete learning process of AMulSys.

Algorithm 1: The learning process of AMulSys

Input: Traces x^t, Logs x^l, Metrics x^m, number of epochs I
Output: Abnormal detection results: Y

```
Data preprocessing
    Trace scheduling relationship extraction, log parsing, log semantic encoding and metric normalization
    Multimodal data alignment
for epoch ← 1 to I do
    Construct Trace-metric graph G^T and Log heterogeneous graph G^L;
    Trace-metric representation: h^tm = GAT(G^T);
    Log representation: h^g = HGTN(G^L);
    Obtain h^{l→tm} and h^{tm→l} via cross-modal attention;
    Compute intra- and inter-contrastive learning loss L_tra, L_ter ;
    Obtain pseudo sample {h̃, ỹ} through category prior guided multimodal data augmentation algorithm;
    Compute inter-sample contrastive learning loss L_orig, L_aug;
    Compute classification loss L_cls;
    // Backward propagation
    Update the whole model by minimizing Eq. (26);
Obtain anomaly detection results;
return Y ;
```

5. Experiment

We conducted the following studies to evaluate the performance of AMulSys: (1) Comparing the fault detection performance of other baseline models with AMulSys; (2) Sensitivity of different key parameters in AMulSys; (3) The impact of different modules in AMulSys on the overall anomaly detection performance.

Table 2
The details of two datasets.

	Services/Nodes	Templates	Variables	%Failures	Modality
GAIA	10	43	83	2.1%	Trace: 2,321,280 Log: 87,974,577 Metric: 734,165
MSDS	5	263	7	1.25%	Trace: 11,157 Log: 130,326 Metric: 325,888

5.1. Experimental setup

5.1.1. Dataset

We evaluate model performance on two real datasets:

GAIA dataset.[1] It is a real microservice system dataset open sourced by CloudWise, which is collected from a simulated microservice system environment consisting of five hosts and 10 microservices, including redisservice1, webservice1, loginservice1 and others. The dataset contains more than 2 million traces data, 87 million logs data and 0.7 million metric data. The operator injects anomalies based on the fault environment in real scenarios, including: cpu_anomalies, file not found and system stuck, etc.

MSDS dataset.[2] It is multimodal monitoring dataset collected from distributed systems (Openstack) for AI-powered analytics. The entire dataset contains information from five physical nodes (Wally113, Wally117, etc.). Among them, there are more than 11,000 traces, more than 130,000 logs, and more than 300,000 metrics. Moreover, the dataset contains three fault injection json files, which contain information such as the start time, end time, and type of the anomaly. Therefore, we can easily obtain the label of each data.

Other details are shown in Table 2, where Templates represents the number of log templates and Variables represents how many metric variables each service or node contains.

5.1.2. Baselines

We compare AMulSys with six baseline methods, which include trace-based unimodal anomaly detection methods: TraceAnomaly [17] constructs a service trace vector and utilizes deep Bayesian networks with posterior flow to analyze the system's normal status. MultimodalTrace [18] transforms traces into spans and response time series, employing multi-modal LSTM models to detect structural anomalies and response time anomalies. Log-based unimodal anomaly detection methods: DeepLog [25] is a classic method based on log index. LogRobust [26] is a well-established method grounded in log semantics. Multimodal-based anomaly detection method: DeepTraLog [7] converts the dependencies between various services into a graph structure. GloVe combined with TF-IDF is used to obtain log semantics and embed them into the graph. A graph-based deep SVDD is utilized for anomaly detection. TraceGra [41] provides a unified graph representation that combines traces and metrics, GAT is employed to analyze the graphs, and then GCN and LSTM are used to extract structural and temporal features, respectively.

5.1.3. Evaluation measurements

This paper adopts Precision, Recall, and F1-score as evaluation measurements, which are defined as follows: $Precision = \frac{TP}{TP+FP}$, $Recall = \frac{TP}{TP+FN}$, $F1 - score = \frac{2 \times Precision \times Recall}{Precision + Recall}$. Where TP is the number of anomalies accurately detected, FP is number of normal samples incorrectly triggering alarms, FN is the number of undetected anomalies. Furthermore, in order to further evaluate the performance of the model under class imbalance, we introduce the ROC curve and

[1] https://github.com/CloudWise-OpenSource/GAIA-DataSet.
[2] https://zenodo.org/records/3549604.

AUC value. Define $FRP = \frac{FP}{TN+FP}$, $TPR = Recall$, where TN is the number of normal samples accurately predicted. The ROC curve is a graph with FPR as the horizontal axis and TPR as the vertical axis. AUC is the area under the ROC curve.

5.1.4. Experiment settings

This experiment is performed on the server with an NVIDIA RTX 3090 GPU, based on Python 3.8.10 and PyTorch 2.0.0+cu118. During the experiment, we select 80% of the data as the training set and the remaining 20% as the test set. In trace-metric representation, there are 2 GAT layers and the hidden size depends on the number of variables of the metric. In log representation, bert-as-service is utilized to generate log semantics. About HGTN, we set the hidden size d_g to 128, stack 1 layer of network, and set the number of attention heads to 4. In the data augmentation process, the *Beta* distribution hyperparameter α is initialized to 1 to make the pseudo samples more evenly distributed in the feature space. This initialization prevents larger λ_l or λ_t from causing the pseudo samples to be biased towards a specific original sample, while smaller values can negatively impact the diversity of the pseudo samples. γ is set to 0.6 to slightly enhance the prominence of the trace-metric representation, which contains two modalities. The batch_size is 64, the epoch size is 50, and Adam is used as the optimizer.

5.2. Evaluation results

The overall experimental results are shown in Table 3. It can be seen that AMulSys outperforms all the baseline models, with the best F1-score 0.974 and 0.976. DeepLog has the lowest detection effect in both datasets, with Pre and Rec of 0.364 and 0.681 in GAIA, and F1-score of 0.457 in MSDS. This is because DeepLog analyzes the index sequence of the log template while disregarding its semantic information. In contrast, LogRobust employs the FastText algorithm to extract semantic and aggregates word vectors based on TF-IDF. During anomaly detection, LogRobust's attention mechanism assigns greater weights to key events, resulting in F1-scores of 26.6% and 27.2% higher than those of DeepLog across the two datasets.

The F1-score of TraceAnomaly is slightly higher than that of MultimodalTrace. While MultimodalTrace utilizes different LSTMs to simultaneously analyze the text labels and response times of normal traces, it overlooks the call relationships between various services. In contrast, TraceAnomaly implements a call path modeling strategy that represents the spans between different services as service trace vectors, achieving a Recall of 0.817 and 0.804, respectively. However, the indicators of the above models are generally lower than those of DeepTraLog, TraceGra, and AMulSys based on multimodal data.

DeepTraLog models traces as a graph structure to preserve the calling relationships between services and incorporates logs based on these traces. It utilizes GloVe to extract log semantic features, which are then used as embeddings for spans within the trace structure graph. Therefore, its F1-score is 23.9% and 32.5% higher than TraceAnomaly and 21.2% and 21.3% higher than LogRobust. TraceGra also models trace as structural graph and integrates both trace and metrics. In contrast to AMulSys, which uses the response time between services as the edge weight of the structural graph, TraceGra concatenates the response times with the metrics to form the feature vector of the span. TraceGra further introduces VGAE and AE to extract their structural and temporal features, respectively, and obtains F1-scores of more than 0.9. In addition, TraceGra outperforms MSDS in the GAIA dataset. This is due to the characteristics of the dataset, where logs are sampled much more frequently than metrics, whereas metrics are sampled quickly in MSDS, which provides a richer representation of service performance. In addition, TraceGra performs better in GAIA than in MSDS. This is due to the characteristics of the dataset. In GAIA, the sampling frequency of logs is much higher than that of metrics, while in MSDS, the sampling frequency of metrics is fast, which can provide a richer representation of service performance.

Table 3
Comparison results on two datasets.

Approach	GAIA			MSDS		
	Precision	Recall	F1-score	Precision	Recall	F1-score
TraceAnomaly	0.635	0.817	0.713	0.501	0.804	0.617
MultimodalTrace	0.577	0.754	0.653	0.609	0.847	0.708
DeepLog	0.364	0.681	0.474	0.346	0.670	0.457
LogRobust	0.710	0.772	0.740	0.722	0.737	0.729
DeepTraLog	0.932	0.973	0.952	0.929	0.955	0.942
TraceGra	0.922	0.976	0.948	0.938	0.964	0.951
AMulSys	0.956	0.994	0.974	0.965	0.988	0.976

In contrast, AMulSys's detection results in both datasets outperform those of DeepTraceLog and TraceGra. This improvement can be attributed to its simultaneous analysis of tri-modal data and the proposed multimodal data augmentation algorithm, which not only address the class-imbalanced problem, but also mitigates the differences in sampling frequency across various modal data to some extent. The multifaceted contrastive learning also optimizes the pseudo sample features. Furthermore, the log heterogeneous graph effectively represents the execution paths between logs in interactive services. The trace-metric graph, which uses metrics as vertex features and response time as edge weights, can more flexibly explore the performance changes between services. Consequently, its F1-score in the GAIA dataset surpasses that of DeepTraceLog by 2.2% and exceeds that of TraceGra by 2.5% in the MSDS.

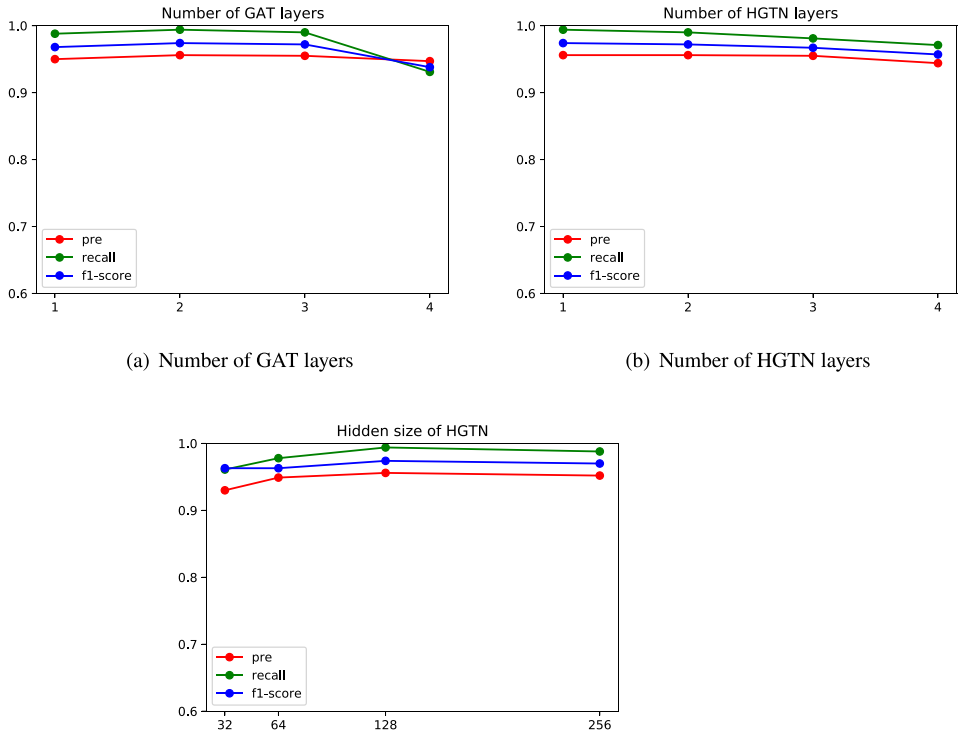
5.3. Parameter sensitivity

In this section, we discuss the impact of various parameter settings on the performance of AMulSys. Specifically, we varied the number of GAT layers, HGTN layers, and the hidden size d_g. When analyzing a specific parameter, we kept all other parameters constant. Parameter sensitivity experiments are conducted on the GAIA dataset, and the results are presented in Fig. 9.

Under various parameter settings, Recall outperforms Precision and F1-score, indicating AMulSys has a low tolerance for anomalies. As shown in Fig. 9(a), the performance is optimal when the number of GAT layers is set to 2, achieving an F1-score of 0.974. When the number of GAT layers is set to 1 or 3, the detection performance is slightly diminished. The reason is that when there is one layer of GAT, each span in the trace obtains limited neighbor information through information transmission and aggregation, and cannot capture the scheduling relationships among service. As the number of GAT layers increases, the model can aggregate deeper neighbor information, but it may learn too much irrelevant information in the metrics of spans, resulting in ineffective distinction of important features. At the same time, the number of model parameters increases, which makes it difficult for the model to converge or even overfit, resulting in poor prediction results. Therefore, when the number of GAT layers exceeds 3, the detection effect decreases significantly.

As illustrated in Fig. 9(b), the F1-score reaches its peak when the number of HGTN layers is set to 1. As the number of layers increases, the effectiveness of anomaly detection gradually diminishes. This indicates that in the GAIA dataset, a single-layer HGTN can stably model the log heterogeneous graph, but as the number of layers increases, model training becomes difficult.

As shown in Fig. 9(c), the performance is poorest when the d_g is set to 32. As the dimension increases, the F1-score gradually improves. When d_g is set to 128, the Recall surpasses 0.99; however, the performance declines as the dimension continues to rise. This indicates that a higher hidden size enhances the model's expressiveness, allowing it to capture more complex contextual information within the log heterogeneous graph. However, excessively high dimensions may lead the HGTN to learn irrelevant information, and the large



(a) Number of GAT layers

(b) Number of HGTN layers

(c) Hidden size of HGTN

Fig. 9. Anomaly detection results for different parameters.

number of parameters will have a negative impact on the convergence speed of the model. Overall, HGTN demonstrates stable performance, indicating that: (1) the log heterogeneous graph effectively reflects the relationship between log events intra- and inter-services in each business process; and (2) we simultaneously model traces, logs, and metrics. Even when the log representation contains noise, the presence of trace-metric features does not directly result in a significant number of misjudgments in AMulSys, further validating the advantages of multimodal data modeling.

5.4. Ablation experiment

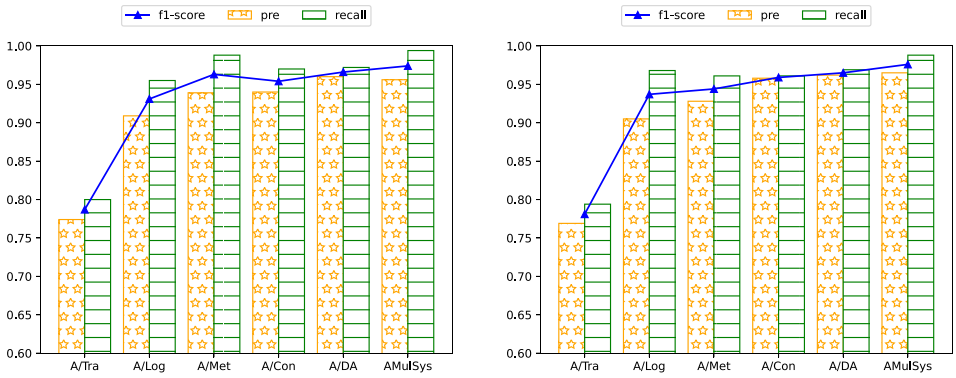
In this section, this paper derives five variant methods based on the original method to verify the contributions of different modules of AMulSys. Specifically, we perform modality ablation. (1) AMulSys/Tra removes trace and only contains log heterogeneous graphs. AMulSys/Log removes log and only contains trace-metric representation. AMulSys/Met removes metric, deletes the metric embedding of each span, and only retains the trace time-related information. (2) AMulSys/Con removes multifaceted contrastive loss. Moreover, we extract 15,000 data from the GAIA dataset for training, and record the training time with and without performing hard sample selection. (3) AMulSys/DA eliminates the multi-modal data augmentation algorithm, conducts anomaly detection after integrating various modal representations.

Result analysis. As illustrated in Fig. 10, the Recall of AMulSys is significantly higher than that of AMulSys/Tra. This discrepancy arises because not all anomalies are captured in the logs, and an over-reliance on a single type of monitoring data can result in the loss of critical information. From a system perspective, various types of monitoring data provide both consistent representations and unique content. Consequently, it is challenging for single modal data to comprehensively reflect the overall system status. The F1-score of AMulSys/Met surpasses that of AMulSys/Tra 17.6% and 16.3% on both datasets, and the F1-score of AMulSys/Log is better than AMulSys/Tra 14.4% and 15.6% on both datasets. However, their various indicators are generally lower than those of AMulSys. This indicates that as multimodality transitions to unimodality, the performance of AMulSys declines significantly. This also indirectly demonstrates the advantages of AMulSys's hierarchical architecture and interactive approach in effectively integrating various monitoring data.

It is worth noting that the F1-score of AMulSys/Tra is still 4% to 30% higher than that of DeepLog and LogRobust, which shows that compared with traditional log sequence and LSTM models, the log heterogeneous graph and HGTN in this paper have unique advantages in analyzing cross-service log events, thereby capturing more anomalies while reducing misjudgments.

The Pre and Recall of AMulSys/Con are significantly higher than those of AMulSys/Tra, further demonstrating that each modal data plays an irreplaceable role in the microservice system. However, its F1-score is lower than that of AMulSys for two reasons. (1) Multifaceted contrastive loss optimizes multimodal features and guarantees data augmentation effects to a certain extent. (2) Its intra-modal contrastive learning enables trace-metric features and log features to adapt to dynamic system environments, and inter-modal contrastive learning obtains modal consistency representation while retaining modal specific information.

Hard sample analysis. We record the training time for 15,000 samples. AMulSys/H represents ignoring hard samples. As shown in the Table 4, we present the training time and average loss for the three cases. It can be observed that AMulSys ($k = 5$) has the shortest training time, followed by AMulSys ($k = 30$), and AMulSys/H has the longest training time. It is worth noting that due to class imbalance, if the k value is too large, it may exceed the actual number of negative or positive sample pairs, so we take $Min(k, M_P)$ and $Min(k, M_N)$. Additionally, the loss is larger with hard samples because the model experiences a higher contrast loss when identifying difficult-to-distinguish instances. This encourages the model to comprehend more complex system status, thereby enhancing its discriminative ability. This demonstrates that this strategy can achieve the dual benefits of enhanced



(a) Ablation experiment results on GAIA (b) Ablation experiment results on MSDS

Fig. 10. Ablation experiment results on different datasets.

detection results and reduced computational overhead without relying on other models.

Data augmentation effect analysis. The multimodal data augmentation strategy plays a crucial role in AMulSys, as evidenced by the fact that the F1-score of AMulSys/DA on both datasets are 0.8% and 1.1% lower than that of AMulSys. This is the detection result under rich data. If there are fewer abnormal data, the effect will drop more obviously. The reason is that our multimodal data augmentation algorithm can expand the number of samples in the feature space, which not only mitigates noise introduced by human involvement but also ensures a diverse range of pseudo-sample features. Furthermore, by incorporating a category prior factor, the likelihood of generating normal pseudo-sample labels is decreased, while the probability of producing abnormal pseudo samples is increased.

In order to further explore the advantages of this data augmentation algorithm, as shown in Table 5, we extract normal and abnormal samples in varying proportions and quantities from the dataset. We also introduce the ROC curve and AUC value, which are sensitive to category, as evaluation indicators.

The results are shown in Fig. 11. First, compared with AMulSys/DA, the ROC curve of AMulSys is farther from the baseline (AUC = 0.5) and has a higher AUC value. In particular, in Fig. 11(b), when the number of samples is small and the class imbalance is more serious, AMulSys is significantly higher than AMulSys/DA. This demonstrates that the data augmentation algorithm presented in this paper enhances detection performance by generating effective pseudo-samples. Second, the AUC value of AMulSys in Fig. 11(c), is the highest. This is because the amount of data is the largest at this time and the ratio of normal and abnormal samples is not much different. AMulSys can employ more raw samples to generate pseudo samples with richer features. Third, comparing Figs. 11(a), and 11(c), we observe that maintaining a constant ratio of normal to abnormal samples, the prediction effect of AMulSys and AMulSys/DA improves as the increase of data. This suggests that the model is able to learn more complex features from more abnormal data, thereby enhancing its ability to classify anomalies. The results presented in Figs. 11(b) and 11(d) further support this conclusion. Final, comparing Figs. 11(a) and 11(d), as the class imbalance increases, the effect of AMulSys/DA decreases significantly, but the AUC value of AMulSys still exceeds that of 0.9. It can still generate some effective pseudo sample features through multimodal data augmentation strategies, and by introducing category factors, it ensures that enough abnormal samples are generated, so that the model can learn discriminative normal and abnormal representations from more samples.

Table 4
Training time and average loss of AMulSys.

	Normal sequences	Abnormal sequences
AMulSys($\varphi = 5$)	1323	0.132
AMulSys($\varphi = 30$)	1341	0.117
AMulSys/H	1370	0.104

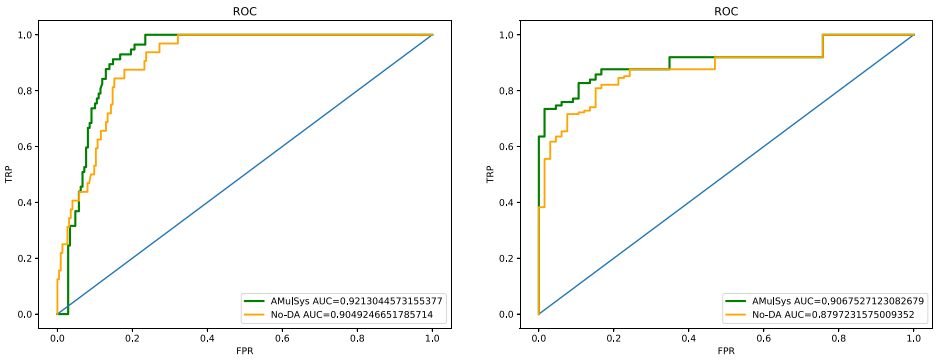
Table 5
Number of normal and abnormal samples.

Ratio	Normal sequences	Abnormal sequences
1:5	7000	1400
1:10		700
1:5	10 000	2000
1:10		1000

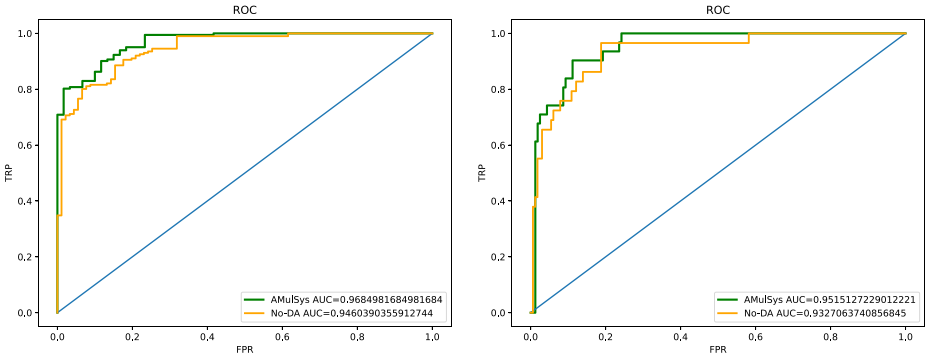
5.5. Robustness evaluation

The robustness of the model is essential to ensure that the system can maintain efficient and accurate anomaly detection capabilities, even in the presence of various complex and dynamically changing operating environments. To evaluate the robustness of AMulSys, we randomly select φ of the multimodal monitoring data for noise injection. For traces, we randomly modify the weights of edges, i.e., the relative waiting times, to simulate network delays or instability between services. For metrics, we randomly select about 1/4 of the variables and inject noise using random numbers. For logs, we conduct random deletions and duplications of log events.

Table 6 presents the experimental results. When φ = 2% noise is injected into the dataset, the Pre of AMulSys on the two datasets decreases by 1.2% and 1%, while the Recall decreases by 0.8% and 0.5%, respectively. This indicates that when a small amount of noise is introduced, the false negative (FN) rate increases, causing the model to misclassify some normal data as abnormal. We can confidently speculate that FN includes a significant amount of noisy data, and that the AMulSys model has a low tolerance for faults. In real-world scenarios, when traces, metrics, or logs exhibit irregular changes, it is prudent to classify them as anomalies. This approach helps prevent the oversight of genuine anomalies that could lead to system failures. In addition, even if φ = 8%, the F1-score of AMulSys is still over 0.95, proving its strong robustness. This is because (1) we employ data augmentation in the multimodal feature space, which eliminates the need for manual intervention and does not rely on a single feature. (2) The multifaceted contrastive learning presented in this paper effectively ensures that the multimodal features can adapt to system changes and enhance the quality of the pseudo samples.



(a) Normal=7000 and Ratio=1:5 (b) Normal=7000 and Ratio=1:10



(c) Normal=10000 and Ratio=1:5 (d) Normal=10000 and Ratio=1:10

Fig. 11. ROC curve under imbalanced data.

Table 6
Robustness evaluation results.

φ	GAIA			MSDS		
	Precision	Recall	F1-score	Precision	Recall	F1-score
0%	0.956	0.994	0.974	0.965	0.988	0.976
2%	0.944	0.986	0.965	0.955	0.983	0.969
8%	0.922	0.985	0.952	0.947	0.979	0.963

6. Discussion

Threats: An inherent threat lies in the accuracy of the baseline model results. Due to some studies not having published code and differences in data collected from different microservice systems, we reproduce the baseline based on understanding of the original article and some open-source implementations to ensure the authenticity and reliability of all models. Some differences may come from different methods of data processing. For fairness, except for special requirements, we uniformly preprocess multimodal monitoring data, including: interpolating missing values with the most recent data, normalization, etc.

Another potential threat arises from the dataset. AMulSys achieves an F1-score exceeding 0.97 on two publicly accessible datasets. However, its robustness in other scenarios remains uncertain. To address this concern, the two datasets selected for this article originate from different microservice systems. GAIA is derived from MicroSS, a business simulation system developed by Cloudwise, while MSDS is sourced from distributed systems (OpenStack). Additionally, the failures represented in both datasets are representative, such as system freezes and network connection failures. In the future, we plan to select additional microservice system datasets to further evaluate AMulSys.

Experience and lesson: AMulSys adopts a hierarchical architecture and introduces trace-metric-based directed weighted graphs and log heterogeneous graphs, which may complicate the reasoning process and affect the efficiency of real-time anomaly detection. Furthermore, the intricate framework may lead to overfitting in small datasets. To address these two issues, we first simplify the model by reducing the number of layers, attention heads, and the hidden size of the GAT or HGTN. Additionally, this paper attains a multimodal data augmentation algorithm that is capable of generating a specified number of pseudo-samples. This approach not only alleviates the issue of class imbalance but also enables the model to adapt to microservice systems of varying sizes to some extent. In the next phase of our research, we will implement incremental learning techniques to update the model in real time, rather than retraining the entire framework each time.

Regarding the selection of evaluation measurements, AMulSys mainly focuses on the comprehensive indicator F1-score to evaluate the effectiveness of anomaly detection. However, in practice, as Precision, Recall, F1-score, and AUC are evaluated from different perspectives and have different characteristics, operators may choose based on business needs. For example, in a train-ticket system, key services such as order management may pay more attention to stability, and have zero tolerance for failures even if false positives are increased, which requires the evaluation to focus on methods that maximize Recall.

7. Conclusion

In this work, we propose AMulSys, a microservice system anomaly detection approach that jointly analyzes trace, log and metric in a framework. AMulSys intuitively models the dependencies between services using a directed acyclic graph and effectively integrates traces and metrics. A graph attention network is employed to derive trace-metric representations. Simultaneously, this paper designs a log heterogeneous graph to capture the execution paths between logs in interactive services. The heterogeneous graph Transformer is used to extract log representations. In order to alleviate the problem of category

imbalance, we extend the Mixup algorithm to multimodal representation and introduce category information to guide pseudo samples to bias towards abnormal classes. Besides, we propose a multifaceted contrastive learning to learn modality-consistent and specific representations from intra-/inter-modal and inter-sample, while also enhancing the effects of data augmentation. During this process, we s select and assign weights to hard samples, which reduces computational overhead without introducing additional parameters.

We compared six baseline models on two real datasets, and AMulSys achieved the highest F1 score, exceeding 0.97. The ablation experiment indicate that different modal data play a crucial role in fault diagnosis outcomes. The analysis of the ROC curve demonstrates that our multimodal data augmentation method effectively addresses the class imbalance issue, resulting in an increase of more than 0.96 in the AUC value. Determining the causes of anomalies can assist operators in repairing faults promptly. In the next work, we will conduct fault classification and root cause analysis based on the results of anomaly detection, with the goal of enhancing the reliability of microservice systems through multimodal and multitask learning.

CRediT authorship contribution statement

Peipeng Wang: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Methodology, Formal analysis, Data curation, Conceptualization. **Xiuguo Zhang:** Writing – review & editing, Writing – original draft, Supervision, Software, Project administration, Methodology, Funding acquisition, Formal analysis, Conceptualization. **Zhiying Cao:** Writing – original draft, Visualization, Funding acquisition, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Grant No. 52231014), Liaoning Province Applied Basic Research Program Project (Grant No. 2023JH2/101300195) and the Cultivation Program for the Excellent Doctoral Dissertation of Dalian Maritime University, China.

Data availability

Data will be made available on request.

References

[1] L. Pham, H. Ha, H. Zhang, Root cause analysis for microservice system based on causal inference: How far are we? in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, Association for Computing Machinery, New York, NY, USA, 2024, pp. 706–715, http://dx.doi.org/10.1145/3691620.3695065.

[2] Y. Sun, Z. Lin, B. Shi, S. Zhang, S. Ma, P. Jin, Z. Zhong, L. Pan, Y. Guo, D. Pei, Interpretable failure localization for microservice systems based on graph autoencoder, ACM Trans. Softw. Eng. Methodol. (2024) http://dx.doi.org/10.1145/3695999, just Accepted.

[3] Z. Xie, C. Pei, W. Li, H. Jiang, L. Su, J. Li, G. Xie, D. Pei, From point-wise to group-wise: A fast and accurate microservice trace anomaly detection approach, in: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Association for Computing Machinery, New York, NY, USA, 2023, pp. 1739–1749, http://dx.doi.org/10.1145/3611643.3613861.

[4] R. Xiao, W. Li, J. Lu, S. Jin, Contexlog: Non-parsing log anomaly detection with all information preservation and enhanced contextual representation, IEEE Trans. Netw. Serv. Manag. 21 (2024) 4750–4762, http://dx.doi.org/10.1109/TNSM.2024.3400283.

[5] Z. Wang, C. Pei, M. Ma, X. Wang, Z. Li, D. Pei, S. Rajmohan, D. Zhang, Q. Lin, H. Zhang, J. Li, G. Xie, Revisiting vae for unsupervised time series anomaly detection: A frequency perspective, in: Proceedings of the ACM Web Conference 2024, Association for Computing Machinery, New York, NY, USA, 2024, pp. 3096–3105, http://dx.doi.org/10.1145/3589334.3645710.

[6] H. Liu, X. Huang, M. Jia, T. Jia, J. Han, Y. Li, Z. Wu, Uac-ad: Unsupervised adversarial contrastive learning for anomaly detection on multi-modal data in microservice systems, IEEE Trans. Serv. Comput. (2024) 1–14, http://dx.doi.org/10.1109/TSC.2024.3411481.

[7] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, D. Zhang, Deeptralog: trace-log combined microservice anomaly detection through graph-based deep learning, in: Proceedings of the 44th International Conference on Software Engineering, Association for Computing Machinery, New York, NY, USA, 2022, pp. 623–634, http://dx.doi.org/10.1145/3510003.3510180.

[8] C. Zhao, M. Ma, Z. Zhong, S. Zhang, Z. Tan, X. Xiong, L. Yu, J. Feng, Y. Sun, Y. Zhang, D. Pei, Q. Lin, D. Zhang, Robust multimodal failure detection for microservice systems, in: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Association for Computing Machinery, New York, NY, USA, 2023, pp. 5639–5649, http://dx.doi.org/10.1145/3580305.3599902.

[9] J. Huang, Y. Yang, H. Yu, J. Li, X. Zheng, Twin graph-based anomaly detection via attentive multi-modal learning for microservice system, in: Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering, IEEE Press, 2024, pp. 66–78, http://dx.doi.org/10.1109/ASE56229.2023.00138.

[10] S. Ying, B. Wang, L. Wang, Q. Li, Y. Zhao, J. Shang, H. Huang, G. Cheng, Z. Yang, J. Geng, An improved knn-based efficient log anomaly detection method with automatically labeled samples, ACM Trans. Knowl. Discov. Data 15 (2021) http://dx.doi.org/10.1145/3441448.

[11] S. Zhang, P. Jin, Z. Lin, Y. Sun, B. Zhang, S. Xia, Z. Li, Z. Zhong, M. Ma, W. Jin, D. Zhang, Z. Zhu, D. Pei, Robust failure diagnosis of microservice system through multimodal data, IEEE Trans. Serv. Comput. 16 (2023a) 3851–3864, http://dx.doi.org/10.1109/TSC.2023.3290018.

[12] V. Petar, C. Guillem, C. Arantxa, R. Adriana, L. Pietro, B. Yoshua, Graph attention networks, in: 6th International Conference on Learning Representations, ICLR, 2018, http://dx.doi.org/10.48550/arXiv.1710.10903.

[13] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: Proceedings of the Web Conference 2020, Association for Computing Machinery, New York, NY, USA, 2020, pp. 2704–2710, http://dx.doi.org/10.1145/3366423.3380027.

[14] R. Lin, H. Hu, Adapt and explore: Multimodal mixup for representation learning, Inf. Fusion 105 (2024) http://dx.doi.org/10.1016/j.inffus.2023.102216.

[15] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, D. Pei, Practical root cause localization for microservice systems via trace analysis, in: 2021 IEEE/ACM 29th International Symposium on Quality of Service, IWQOS, IEEE, 2021, pp. 1–10, http://dx.doi.org/10.1109/IWQOS52092.2021.9521340.

[16] G. Yu, P. Chen, H. Chen, Z. Guan, Z. Huang, L. Jing, T. Weng, X. Sun, X. Li, Microrank: End-to-end latency issue localization with extended spectrum analysis in microservice environments, in: Proceedings of the Web Conference 2021, Association for Computing Machinery, New York, NY, USA, 2021, pp. 3087–3098, http://dx.doi.org/10.1145/3442381.3449905.

[17] P. Liu, H. Xu, Q. Ouyang, R. Jiao, Z. Chen, S. Zhang, J. Yang, L. Mo, J. Zeng, W. Xue, D. Pei, Unsupervised detection of microservice trace anomalies through service-level deep bayesian networks, in: 2020 IEEE 31st International Symposium on Software Reliability Engineering, ISSRE, IEEE, 2020, pp. 48–58, http://dx.doi.org/10.1109/ISSRE5003.2020.00014.

[18] S. Nedelkoski, J. Cardoso, O. Kao, Anomaly detection from system tracing data using multimodal deep learning, in: 2019 IEEE 12th International Conference on Cloud Computing, CLOUD, IEEE Press, 2019, pp. 179–186, http://dx.doi.org/10.1109/CLOUD.2019.00038.

[19] Z. Xie, H. Xu, W. Chen, W. Li, H. Jiang, L. Su, H. Wang, D. Pei, Unsupervised anomaly detection on microservice traces through graph vae, in: Proceedings of the ACM Web Conference 2023, Association for Computing Machinery, New York, NY, USA, 2023, pp. 2874–2884, http://dx.doi.org/10.1145/3543507.3583215.

[20] S. Zhang, Z. Pan, H. Liu, P. Jin, Y. Sun, Q. Ouyang, J. Wang, X. Jia, Y. Zhang, H. Yang, Y. Zou, D. Pei, Efficient and robust trace anomaly detection for large-scale microservice systems, in: 2023 IEEE 34th International Symposium on Software Reliability Engineering, ISSRE, 2023, pp. 69–79, http://dx.doi.org/10.1109/ISSRE59848.2023.00012.

[21] S. Kong, J. Ai, M. Lu, Y. Gong, Grand: Gan-based software runtime anomaly detection method using trace information, Neural Netw. 169 (2024) 365–377, http://dx.doi.org/10.1016/j.neunet.2023.10.036.

[22] P. He, J. Zhu, Z. Zheng, M.R. Lyu, Drain: An online log parsing approach with fixed depth tree, in: 2017 IEEE International Conference on Web Services, ICWS, IEEE, 2017, http://dx.doi.org/10.1109/ICWS.2017.13.

[23] X. Li, P. Chen, L. Jing, Z. He, G. Yu, Swisslog: Robust anomaly detection and localization for interleaved unstructured logs, IEEE Trans. Dependable Secur. Comput. 20 (2023) 2762–2780, http://dx.doi.org/10.1109/TDSC.2022.3162857.

[24] A.A. Makanju, A.N. Zincir-Heywood, E.E. Milios, Clustering event logs using iterative partitioning, in: Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Association for Computing Machinery, New York, NY, USA, 2009, pp. 1255–1264, http://dx.doi.org/10.1145/1557019.1557154.

[25] M. Du, F. Li, G. Zheng, V. Srikumar, Deeplog: Anomaly detection and diagnosis from system logs through deep learning, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, Association for Computing Machinery, New York, NY, USA, 2017, pp. 1285–1298, http://dx.doi.org/10.1145/3133956.3134015.

[26] X. Zhang, Y. Xu, Q. Lin, B. Qiao, H. Zhang, Y. Dang, C. Xie, X. Yang, Q. Cheng, Z. Li, J. Chen, X. He, R. Yao, J.G. Lou, M. Chintalapati, F. Shen, D. Zhang, Robust log-based anomaly detection on unstable log data, in: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Association for Computing Machinery, New York, NY, USA, 2019, pp. 807–817, http://dx.doi.org/10.1145/3338906.3338931.

[27] H. Guo, S. Yuan, X. Wu, Logbert: Log anomaly detection via bert, in: 2021 International Joint Conference on Neural Networks, IJCNN, IEEE Press, 2021, pp. 1–8, http://dx.doi.org/10.1109/IJCNN52387.2021.9534113.

[28] J. Qi, Z. Luan, S. Huang, C. Fung, H. Yang, H. Li, D. Zhu, D. Qian, Logencoder: Log-based contrastive representation learning for anomaly detection, IEEE Trans. Netw. Serv. Manag. 20 (2023) 1378–1391, http://dx.doi.org/10.1109/TNSM.2023.3239522.

[29] Y. Fu, K. Liang, J. Xu, Mlog: Mogrifier lstm-based log anomaly detection approach using semantic representation, IEEE Trans. Serv. Comput. 16 (2023) 3537–3549, http://dx.doi.org/10.1109/TSC.2023.3289488.

[30] P. Wang, X. Zhang, Z. Cao, W. Xu, W. Li, Loggt: Cross-system log anomaly detection via heterogeneous graph feature and transfer learning, Expert Syst. Appl. 251 (2024a) 124082, http://dx.doi.org/10.1016/j.eswa.2024.124082.

[31] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, D. Pei, Robust anomaly detection for multivariate time series through stochastic recurrent neural network, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, Association for Computing Machinery, New York, NY, USA, 2019, pp. 2828–2837, http://dx.doi.org/10.1145/3292500.3330672.

[32] J. Xu, H. Wu, J. Wang, Long Mingsheng, Anomaly transformer: Time series anomaly detection with association discrepancy, in: The Tenth International Conference on Learning Representations, ICLR, 2022, pp. 6894–6910, http://dx.doi.org/10.48550/arXiv.2110.02642.

[33] G. Li, J.J. Jung, Deep learning for anomaly detection in multivariate time series: Approaches, applications, and challenges, Inf. Fusion 91 (2023) 93–102, http://dx.doi.org/10.1016/j.inffus.2022.10.008.

[34] H. Zhao, Y. Wang, J. Duan, C. Huang, D. Cao, Y. Tong, B. Xu, J. Bai, J. Tong, Q. Zhang, Multivariate time-series anomaly detection via graph attention network, in: 2020 IEEE International Conference on Data Mining, ICDM, IEEE Press, 2020, pp. 841–850, http://dx.doi.org/10.1109/ICDM50108.2020.00093.

[35] H. Zhu, S. Rho, S. Liu, F. Jiang, Learning spatial graph structure for multivariate kpi anomaly detection in large-scale cyber–physical systems, IEEE Trans. Instrum. Meas. 72 (2023) 1–16, http://dx.doi.org/10.1109/TIM.2023.3284920.

[36] L. Zhou, H. Wang, Mst-gat: A multi-perspective spatial–temporal graph attention network for multi-sensor equipment remaining useful life prediction, Inf. Fusion 110 (2024) 102462, http://dx.doi.org/10.1016/j.inffus.2024.102462.

[37] Y. Sun, X. Wang, D. Peng, Z. Ren, X. Shen, Hierarchical hashing learning for image set classification, IEEE Trans. Image Process. 32 (2023) 1732–1744, http://dx.doi.org/10.1109/TIP.2023.3251025.

[38] N. Zhao, J. Chen, Z. Yu, H. Wang, J. Li, B. Qiu, H. Xu, W. Zhang, K. Sui, D. Pei, Identifying bad software changes via multimodal anomaly detection for online service systems, in: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Association for Computing Machinery, New York, NY, USA, 2021, pp. 527–539, http://dx.doi.org/10.1145/3468264.3468543.

[39] L. Zheng, Z. Chen, J. He, H. Chen, Mulan: Multi-modal causal structure learning and root cause analysis for microservice systems, in: Proceedings of the ACM Web Conference 2024, Association for Computing Machinery, New York, NY, USA, 2024, pp. 4107–4116, http://dx.doi.org/10.1145/3589334.3645442.

[40] C. Lee, T. Yang, Z. Chen, Y. Su, Y. Yang, M.R. Lyu, Heterogeneous anomaly detection for software systems via semi-supervised cross-modal attention, in: 2023 IEEE/ACM 45th International Conference on Software Engineering, ICSE, IEEE Press, 2023, pp. 1724–1736, http://dx.doi.org/10.1109/ICSE48619.2023.00148.

[41] J. Chen, F. Liu, J. Jiang, G. Zhong, D. Xu, Z. Tan, S. Shi, Tracegra: A trace-based anomaly detection for microservice using graph deep learning, Comput. Commun. 204 (2023) 109–117, http://dx.doi.org/10.1016/j.comcom.2023.03.028.

[42] Y. Sun, B. Shi, M. Mao, M. Ma, S. Xia, S. Zhang, D. Pei, Art: A unified unsupervised framework for incident management in microservice systems, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, Association for Computing Machinery, New York, NY, USA, 2024, pp. 1183–1194, http://dx.doi.org/10.1145/3691620.3695495.

[43] S. Zhang, Y. Zhao, S. Xia, S. Wei, Y. Sun, C. Zhao, S. Ma, J. Kuang, B. Zhu, L. Pan, Y. Guo, D. Pei, No more data silos: Unified microservice failure diagnosis with temporal knowledge graph, IEEE Trans. Serv. Comput. (2024) 1–14, http://dx.doi.org/10.1109/TSC.2024.3489444.

[44] C. Lee, T. Yang, Z. Chen, Y. Su, M.R. Lyu, Eadro: An end-to-end troubleshooting framework for microservices on multi-source data, in: Proceedings of the 45th International Conference on Software Engineering, IEEE Press, 2023a, pp. 1750–1762, http://dx.doi.org/10.1109/ICSE48619.2023.00150.

[45] J. Devlin, M.W. Chang, K. Lee, T. Kristina, Bert: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT, Association for Computational Linguistics, 2019, pp. 4171–4186, http://dx.doi.org/10.18653/v1/n19-1423.

[46] J.W. Wei, K. Zou, EDA: easy data augmentation techniques for boosting performance on text classification tasks, in: K. Inui, J. Jiang, V. Ng, X. Wan (Eds.), Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019, Association for Computational Linguistics, 2019, pp. 6381–6387, http://dx.doi.org/10.18653/V1/D19-1670.

[47] H. Zhang, Y.N. Cissé, D. Lopez-Paz, Mixup: Beyond empirical risk minimization, in: 6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3 2018, Conference Track Proceedings, OpenReview.net, 2018, URL http://arxiv.org/abs/1710.09412.

[48] Y. Xian, W. Chen, Y. Zhang, H. Wang, Category prior guided mixup data argumentation for charge prediction, Acta Automat. Sinica 48 (2022) http://dx.doi.org/10.16383/j.aas.c200908.

[49] M. Hong, J. Choi, G. Kim, Stylemix: Separating content and style for enhanced data augmentation, in: 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2021, pp. 14857–14865, http://dx.doi.org/10.1109/CVPR46437.2021.01462.

[50] S. Mai, Y. Zeng, H. Hu, Learning from the global view: Supervised contrastive learning of multimodal representation, Inf. Fusion 100 (2023) http://dx.doi.org/10.1016/j.inffus.2023.101920.

[51] Y. Sun, Z. Ren, P. Hu, D. Peng, X. Wang, Hierarchical consensus hashing for cross-modal retrieval, IEEE Trans. Multimed. 26 (2024c) 824–836, http://dx.doi.org/10.1109/TMM.2023.3272169.

[52] T.T. Cai, J. Frankle, D.J. Schwab, A.S. Morcos, Are all negatives created equal in contrastive instance discrimination?, 2020, CoRR abs/2010.06682 URL https://arxiv.org/abs/2010.06682 arXiv:2010.06682.

[53] Y. Liu, X. Yang, S. Zhou, X. Liu, Z. Wang, K. Liang, W. Tu, L. Li, J. Duan, C. Chen, Hard sample aware network for contrastive deep graph clustering, Proc. AAAI Conf. Artif. Intell. 37 (2023) 8914–8922, http://dx.doi.org/10.1609/aaai.v37i7.26071.

[54] J. Yin, H. Wu, S. Sun, Effective sample pairs based contrastive learning for clustering, Inf. Fusion 99 (2023) http://dx.doi.org/10.1016/j.inffus.2023.101899.

[55] Q. Wang, X. Zhang, X. Wang, Z. Cao, Log sequence anomaly detection method based on contrastive adversarial training and dual feature extraction, Entropy 24 (2022) http://dx.doi.org/10.3390/e24010069.